

Lab Sheet: Encapsulation

Course: Introduction to Programming Language II (Java)

Topic: Encapsulation, Access Modifiers, Data Hiding, Getters and Setters

Level: Beginner to Intermediate

Duration: 3 hours

Prepared by: Ashraful Islam Paran, Dept. of CSE, UIU

1. Objectives

By the end of this lab, students will be able to:

- Understand the concept of Encapsulation in OOP.
- Apply access modifiers (`private`, `public`, `protected`, `default`) effectively.
- Implement data hiding using `private` fields.
- Use getter and setter methods to control access to data.
- Create immutable classes using encapsulation.
- Understand the benefits of encapsulation (data integrity, maintainability, security).

2. What is Encapsulation?

Encapsulation is one of the four fundamental OOP principles. It is the mechanism of **wrapping the data (variables) and code (methods)** together as a single unit.

In Java, encapsulation is achieved by:

- Declaring class variables as `private`.
- Providing `public` getter and setter methods to access and update the data.

Key Benefits:

- Data Hiding: Prevents direct access to internal data.
- Better Control: Validation logic can be added in setters.
- Loose Coupling: Reduces dependency between classes.
- Improved Maintainability and Security.

3. Example 1: Basic Encapsulation

```
class Student {  
    // private fields - data hiding  
    private String name;  
    private int age;  
  
    // Getter methods  
    public String getName() {
```

```
        return name;
    }

    public int getAge() {
        return age;
    }

    // Setter methods with validation
    public void setName(String name) {
        this.name = name;
    }

    public void setAge(int age) {

        this.age = age;
    }
}

public class EncapsulationDemo {
    public static void main(String[] args) {
        Student s = new Student();
        s.setName("Alice");
        s.setAge(20);

        System.out.println("Name: " + s.getName());
        System.out.println("Age: " + s.getAge());
    }
}
```

Output:

Name: Alice

Age: 20

4. Example 2: Bank Account (Real-world Example)

```
class BankAccount {
    private String accountNumber;
    private double balance;

    public BankAccount(String accNum, double initialBalance) {
        this.accountNumber = accNum;
        if (initialBalance >= 0) {
            this.balance = initialBalance;
        }
    }

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited: " + amount);
        }
    }

    public boolean withdraw(double amount) {
        if (amount > 0 && balance >= amount) {
            balance -= amount;
            System.out.println("Withdrawn: " + amount);
            return true;
        }
        System.out.println("Insufficient balance or invalid amount!");
        return false;
    }
}

public class BankDemo {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount("ACC12345", 5000);
        System.out.println("Current Balance: " + acc.getBalance());
        acc.deposit(2000);
        acc.withdraw(1000);
        System.out.println("Final Balance: " + acc.getBalance());
    }
}
```

Output:

```
Current Balance: 5000.0
Deposited: 2000.0
Withdrawn: 1000.0
Final Balance: 6000.0
```

5. Example 3: Immutable Class

```
final class ImmutableStudent {
    private final String name;
    private final int roll;

    public ImmutableStudent(String name, int roll) {
        this.name = name;
        this.roll = roll;
    }

    public String getName() {
        return name;
    }

    public int getRoll() {
        return roll;
    }
}

public class ImmutableDemo {
    public static void main(String[] args) {
        ImmutableStudent s = new ImmutableStudent("Bob", 101);
        System.out.println(s.getName() + " - " + s.getRoll());
        // s.name = "Something"; // Error: final field
    }
}
```

6. Access Modifiers Summary

- private → Accessible only within the class
- Default (no modifier) → Accessible within the package
- protected → Accessible within package and subclasses
- public → Accessible from everywhere

Task 2.1 – Basic Encapsulation (10 min)

- a. Create a class Employee with private fields: name (String), id (int), salary (double). Add appropriate getters and setters. Add validation in setter for salary (must be positive).

Sample Code:

```
class Employee {  
    private String name;  
    private int id;  
    private double salary;  
  
    // Add getters and setters here  
    public void setSalary(double salary) {  
        if (salary > 0) this.salary = salary;  
    }  
  
    public double getSalary() {  
        return salary;  
    }  
    // ... complete other getters/setters  
}
```

Task 2.2 – Student Management System (15 min)

- a. Create a Student class with private fields: name, age, grade.
- b. Add a constructor and getter/setter methods.
- c. Add a method promote() that increases grade by 1 (max 12).

Task 2.3 – Library Book Class (15 min)

- a. Design a Book class with private fields: title, author, isbn, isAvailable (boolean).
- b. Implement methods: borrowBook() and returnBook().
- c. Ensure a book cannot be borrowed if not available.

Task 2.4 – Immutable Circle Class (10 min)

- a. Create an immutable class Circle with private final radius.
- b. Add methods to calculate area and circumference.
- c. Make the class final.

Sample Code Snippet:

```
final class Circle {  
    private final double radius;  
  
    public Circle(double radius) {  
        this.radius = Math.max(radius, 0);  
    }  
  
    public double getArea() {  
        return Math.PI * radius * radius;  
    }  
}
```

Keep Coding! Practice Encapsulation to write secure and maintainable Java code.